

Trees Study Guide

Trees Study Guide

1. Overview

Trees are hierarchical data structures composed of nodes connected by edges. They are essential in DSA because they model relationships, enable efficient searching, and support recursive problem-solving. Many LeetCode problems rely on understanding traversal patterns, recursion, and structural properties.

Common tree types:

- Binary Trees
- Binary Search Trees (BST)
- N-ary Trees
- Tries
- Heaps (as arrays)

Trees are ideal when you need:

- Hierarchical representation
- Fast lookup (BST)
- Structured recursion
- Parent-child relationships

2. Core Tree Traversal Patterns

2.1 [Depth-First Search \(DFS\)](#)

DFS explores as far as possible down each branch.

Forms:

- Preorder (root → left → right)
- Inorder (left → root → right)
- Postorder (left → right → root)

Example (recursive):

```
def inorder(root):  
    if not root: return  
    inorder(root.left)  
    print(root.val)  
    inorder(root.right)
```

2.2 [Breadth-First Search \(BFS\)](#)

Uses a queue to explore level by level.

Examples:

- Level Order Traversal
- Zigzag Level Order

Key idea:

```
queue = [root]
while queue:
    for _ in range(len(queue)):
        node = queue.pop(0)
    ...
```

2.3 [Binary Search Tree \(BST\) Properties](#)

BST invariant:

```
left subtree < root < right subtree
```

This enables:

- $O(\log n)$ search
- Ordered traversal via inorder

3. Essential Tree Techniques

3.1 Recursion Templates

Most tree problems follow this pattern:

```
def dfs(node):
    if not node:
        return base_case

    left = dfs(node.left)
    right = dfs(node.right)

    return combine(left, right, node)
```

3.2 Height / Depth Computation

```
def height(root):
    if not root: return 0
    return 1 + max(height(root.left), height(root.right))
```

Used in:

- Balanced Binary Tree
 - Diameter of Binary Tree
-

3.3 Lowest Common Ancestor (LCA)

```
def lca(root, p, q):  
    if not root or root in (p, q): return root  
    left = lca(root.left, p, q)  
    right = lca(root.right, p, q)  
    if left and right: return root  
    return left or right
```

3.4 Serialization / Deserialization

Used for tree encoding.

Example (BFS):

```
# Serialize  
queue = [root]  
res = []  
while queue:  
    node = queue.pop(0)  
    if node:  
        res.append(str(node.val))  
        queue += [node.left, node.right]  
    else:  
        res.append("null")
```

4. Example Problem: Maximum Depth of Binary Tree

```
def maxDepth(root):  
    if not root: return 0  
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

Why recursion works:

- Depth of a tree = 1 + max depth of children.
-

5. Example Problem: Validate BST

```
def isValidBST(root):  
    def dfs(node, low, high):  
        if not node: return True  
        if not (low < node.val < high): return False  
        return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)  
    return dfs(root, float('-inf'), float('inf'))
```

6. Common Pitfalls

- Forgetting base cases in recursion
 - Confusing depth vs height
 - Incorrect BST boundary handling
 - Using BFS when DFS is simpler (or vice versa)
 - Not considering null children in serialization
-

7. Practice Roadmap

Beginner

- Maximum Depth of Binary Tree
- Invert Binary Tree
- Symmetric Tree

Intermediate

- Binary Tree Level Order Traversal
- Validate BST
- LCA of Binary Tree

Advanced

- Serialize/Deserialize Binary Tree
 - Diameter of Binary Tree
 - Path Sum II
 - Recover BST
-

8. Key Takeaways

- Trees rely heavily on recursion and traversal patterns.
- BSTs provide ordered structure for efficient search.
- Many problems reduce to DFS with a clear base case and combination rule.

End of document.